Decision-making with Uncertainty

Kaiqi Zhao

Harbin Institute of Technology, Shenzhen



Decision-making with Uncertainty

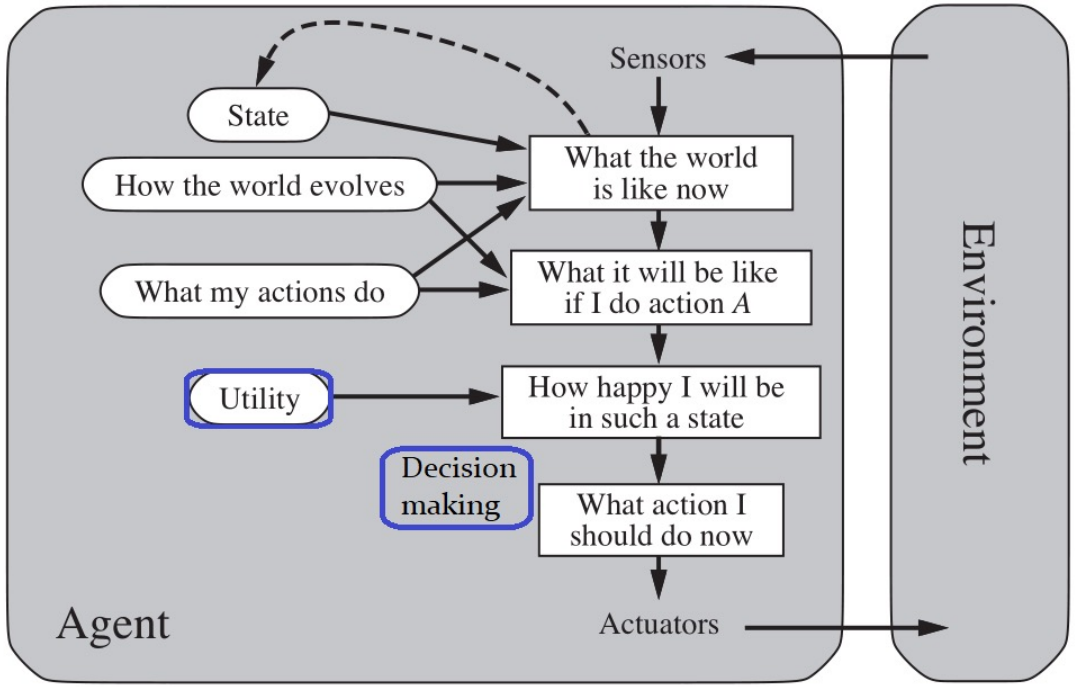
Basic tools:

- Probability theory: quantifying uncertainty and calculating expectations
- Bayesian networks: reason about probabilistic outcomes with imperfect information.
- Markov model: reason about discrete time and sequential actions.
- Utility theory: to be covered in this lecture.

Evaluating the desirability of (probabilistic) outcomes:

- The agent hopes to maximise the chance of meeting its goal, i.e., arrive at outcomes that are more desirable.
- **Preferences**: How desirable an outcome is.
- **Utility function**: A numerical value that expresses the agent's preference of a state

Utility-based agent.



哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

Utility Theory

Question: Can preferences be represented by numbers?

Example 1. Suppose you have $1000 in savings. Would you risk $1000 for a 10% chance to win $9000?

- Option 1: Keep $1000. Expected win: 0.
- Option 2: 10% chance of winning $9000, but 90% chance of losing $1000. Expected win: $0.1\times 9000+0.9\times(-1000)=0$

Example 2. Would you risk $1 to enter a lottery with a 0.1% chance to win $999?

- Option 1: Do not enter the lottery. Expected win: 0.
- Option 2: 0.1% chance of winning $999, but 99.9% chance of losing $1. Expected win: $0.001\times 999+0.999\times(-1)=0$.

Utility theory is the study of utilities and their relations to preferences over uncertain outcomes

Definition

- A lottery is a finite distribution over a set $\{a_1, \ldots, a_k\}$, written as
$$[p_1: a_1, p_2: a_2, \ldots, p_k: a_k]$$
where $p_i \in \mathbb{R}$ is non-negative with $\sum_i p_i = 1$
- The set of outcomes Ω contains all lotteries over states S, and all lotteries over outcomes.

- Define the weak preference relation $\succcurlyeq$:

  $o_1 \succcurlyeq o_2$ if outcome o_1 is at least as desirable as outcome o_2

- Define the indifference relation $\sim$:

  $o_1 \sim o_2$ if $o_1 \succcurlyeq o_2$ and $o_2 \succcurlyeq o_1$, i.e., o_1 and o_2 are equally preferred

- Define the strict preference relation $\succ$:

  $o_1 \succ o_2$ if $o_1 \succcurlyeq o_2$ and $o_2 \not\succcurlyeq o_1$, i.e., o_1 and o_2 are not equally preferred

Axioms for Rationality

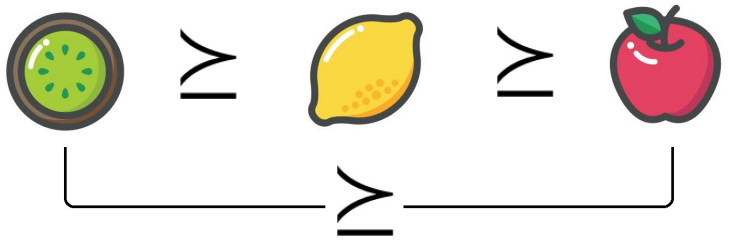
Axioms defining the preferences of a rational agent:

- Axiom 1 Completeness. An agent has preference between all pairs of outcomes:

$$o_1 \succcurlyeq o_2 \text{ or } o_2 \succcurlyeq o_1$$

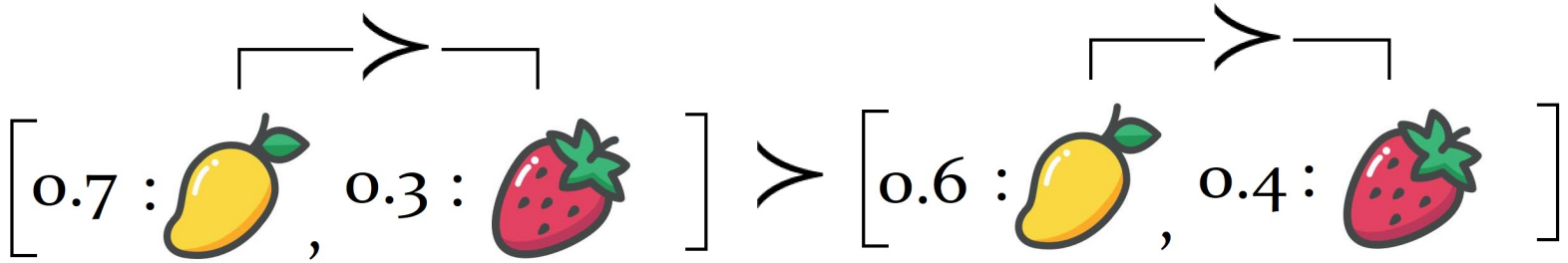
- Axiom 2 Transitivity. Preference must be transitive:

$$o_1 \succcurlyeq o_2 \,\&\, o_2 \succcurlyeq o_3 \Rightarrow o_1 \succcurlyeq o_3$$



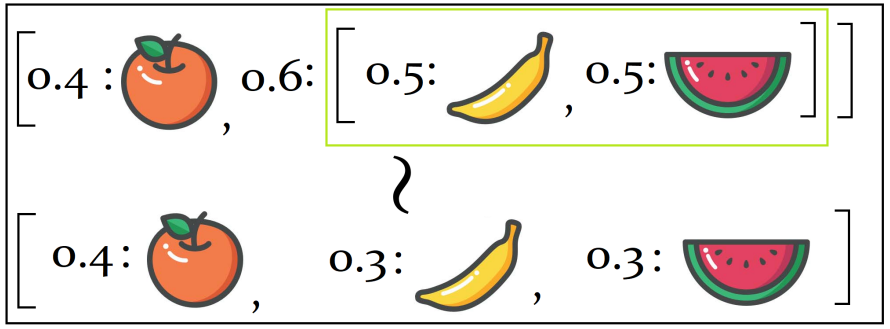
- Axiom 3 Monotonicity. An agent prefers a larger chance of getting a better outcome than a smaller chance of getting the better outcome.

$$\text{if } o_1 \succcurlyeq o_2 \,\&\, p > q, \text{ then } [p{:}\, o_1, (1-p){:}\ o_2] \succ [q{:}\, o_1, (1-q){:}\, o_2]$$



- Axiom 4 Decomposability. An agent is indifferent between lotteries that have the same probabilities over the same outcomes. E.g.,

$$[p: o_1, (1-p): [q: o_2, (1-q): o_3]] \sim [p: o_1, (1-p)q: o_2, (1-p)(1-q): o_3]$$





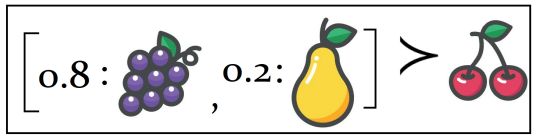
- Axiom 5 Continuity. Suppose $o_1 \succ o_2 \succ o_3$. Then there exists $p \in [0,1]$ such that

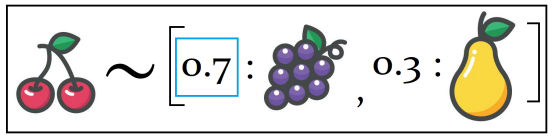
$$o_2 \sim [p: o_1, (1-p): o_3]$$

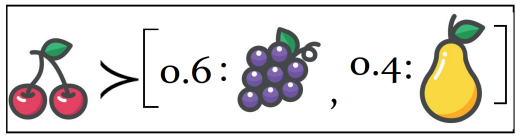
Note. For p above, by monotonicity and transitivity:

$$\forall p' > p:\ [p': o_1, (1-p'): o_3] \succ o_2 \text{ and } \forall p'' < p: o_2 \succ [p'': o_1, (1-p''): o_3]$$





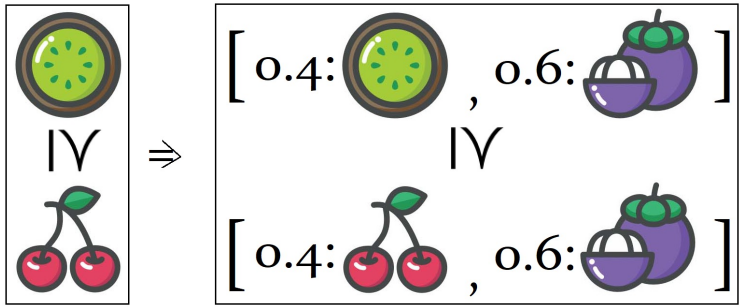




- Axiom 6 Independence. If $o_1 \succcurlyeq o_2$, then for any number $p \in [0,1]$ and outcome o_3:

$$[p: o_1, (1-p): o_3] \succcurlyeq [p: o_2, (1-p): o_3]$$

Note. If $o_1 \sim o_2$, then for any $p \in [0,1]$ and o_3, $[p: o_1, (1-p): o_3] \sim [p: o_2, (1-p): o_3]$.



VNM rationality

Definition

An agent is defined to be (VNM-)rational if it obeys the completeness, transitivity, monotonicity, decomposability, continuity, and independence axioms.

Theorem (von Neumann and Morgenstern 1947)

If an agent is rational, then for every outcome o_i, there is a real number $u(o_i)$ such that

- (1) $o_i \succ o_j$ if and only if $u(o_i) > u(o_j)$ and
- (2) Utilities are linear with probabilities:

$$u([p_1: o_1, p_2: o_2, \ldots, p_k: o_k]) = p_1 u(o_1) + p_2 u(o_2) + \cdots + p_k u(o_k)$$

Consequence of VNM rationality.

- Although preferences may seem to be complex and multifaceted, a rational agent's value for an outcome can be measure by a (one-dimensional) number $u(o_i)$, i.e., the utility of the outcome o_i.
- The utility of a (probabilistic) outcome $[p_1: o_1, p_2: o_2, \dots, p_k: o_k]$ can be described as the linear sum, i.e., the expected utility:

$$p_1 u(o_1) + p_2 u(o_2) + \cdots + p_k u(o_k)$$

- Linear scalability: Suppose $u: \Omega \to \mathbb{R}$ is a utility function. Then for any constant $c > 0, u': \Omega \to \mathbb{R}$ defined by $u'(o) = cu(o)$ is also a utility function.

Expected utility hypothesis: The utility function correctly reflects the performance measure of an agent, i.e., if the agent acts so as to maximize the expected utility, then the agent will achieve the highest performance.

Example. Suppose you have \$1000 in savings. Would you risk \$1000 for a 10% chance to win \$9000?

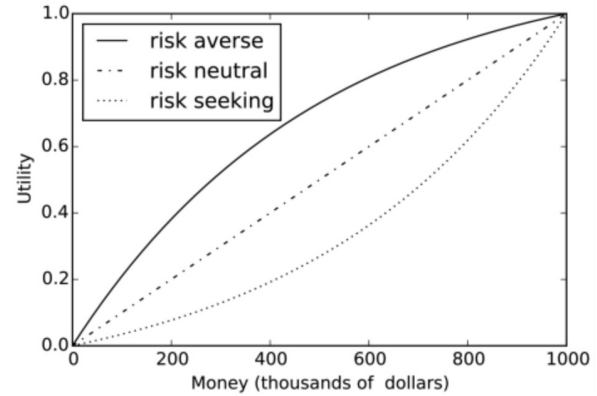
- Outcome o_0: Not participating, \$0
- Outcome o_{9000}: Participating, \$9000
- Outcome o_{-1000}: Participating, -\$1000

Then, the agent has lotteries:

- No participating: [1:o_0, 0:o_{9000}, 0:o_{-1000}], **utility**: $u(o_0)$
- Participating: [0:o_0, 0.1:o_{9000}, 0.9:o_{-1000}], **utility**: $0.1u(o_{9000}) + 0.9u(o_{-1000})$

Monotonicity Assumption: $o_{9000} \succ o_0 \succ o_{-1000}$.

- $u(o_0) > 0.1u(o_{9000}) + 0.9u(o_{-1000})$: the agent is risk averse
- $u(o_0) = 0.1u(o_{9000}) + 0.9u(o_{-1000})$: the agent is risk neutral
- $u(o_0) < 0.1u(o_{9000}) + 0.9u(o_{-1000})$: the agent is risk seeking



Multi-dimensional Utility

Question.

- Atomic representation. A utility function is defined on each state $s \in S$
- Factored representation. How to extend the utility function to factored representations over variables?

Example [warrior]. What items should be warrior choose?

- Weapon: Sword, Wooden club
- Armor: Chain vest, Cloth armor

Additive independence property: A state is described by k variables $V_1, \dots, V_k$:

- A utility function u_i: $\text{dom}(V_i) \to [0,1]$ is defined for any variable $V_i, 1 \leq i \leq k$.
- For state $s = (V_1 = v_1, \dots, V_k = v_k)$, $u(s) = w_1 u_1(v_1) + \cdots + w_k u_k(v_k)$ for weights $w_1, \dots, w_k \in \mathbb{R}$

E.g., $u(\text{Weapon, Armor}) = w_1 u_1(\text{Weapon}) + w_2 u_2(\text{Armor})$

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

One-off Decision Problems

One-off Decision Problem

One-off Decision Problem

- $V_1, \dots, V_k$: the set of decision variables
- $R_1, \dots, R_l$: the set of random variables
- $S = \prod_{i=1}^{k} \mathrm{dom}(V_i) \times \prod_{j=1}^{l} \mathrm{dom}(R_j)$: the search space
- $P(R_1, \dots, R_l | V_1, \dots, V_k)$: the conditional probability of random variables given decisions.
- $u(s)$: S $\rightarrow \mathbb{R}$: Preferences/utility function over S.
- Goal: Choose values for $V_1, \dots, V_k$ to maximise the expected utility

$$\mathrm{argmax}_{v_1,\dots,v_k} \sum_{r_1,\dots,r_l} P(r_1, \dots, r_l | v_1, \dots, v_k) \cdot u(v_1, \dots, v_k, r_1, \dots, r_l)$$

One-off decision problem can be described by two types of representations:

- **Decision tree**: factored representation
- **Decision network**: structured representation

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

Example [delivery robot]. Consider a delivery robot aiming to move to a target location

- Variable 1 $Acc \in \{0,1\}$: The robot may encounter an accident
- Variable 2 $Pad \in \{0,1\}$: The robot can choose to put on a pad, which avoids severe damage, but increases weight.
- Variable 3 $Way \in \{0,1\}$: The robot can choose to use either a short or a long route. The short one has the higher chance of encountering accident.

The variables satisfy the following:

- The utility depends on all three variables.
- The value of Acc depends on Way, i.e., $P(Acc, Way) \neq P(Acc)P(Way)$
- The effect of Acc on the utility is different for different values for Pad



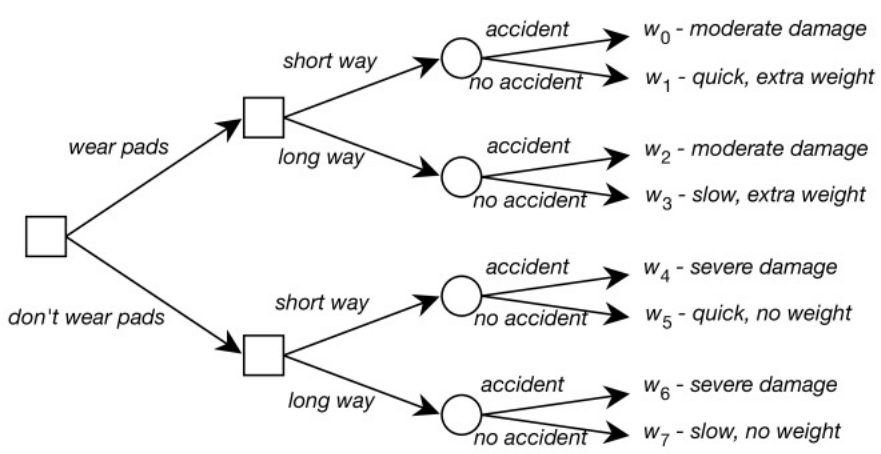
Utility function: $u: S \rightarrow \mathbb{R}$

Pad	Way	Acc	Outcome	$u(Pad, Way, Acc)$
0	0	0	w_5	100
0	0	1	w_4	3
0	1	0	w_7	80
0	1	1	w_6	0
1	0	0	w_1	95
1	0	1	w_0	35
1	1	0	w_3	75
1	1	1	w_2	30

Representation 1: Decision trees

A decision tree connects variables with outcomes, but not dependencies between variables.

Example. [delivery robot]





Conditional probability:

Way	Acc	$\mathbf{P}(Acc \mid Way)$
0	0	0.8
0	1	0.2
1	0	0.99
1	1	0.01

Utility function: $u : S \to \mathbb{R}$

Pad	Way	Acc	Outcome	$u(Pad, Way, Acc)$
0	0	0	w_5	100
0	0	1	w_4	3
0	1	0	w_7	80
0	1	1	w_6	0
1	0	0	w_1	95
1	0	1	w_0	35
1	1	0	w_3	75
1	1	1	w_2	30

Expected utility: $E(u \mid Pad, Way)$

Pad	Way	$E(u \mid Pad, Way)$
0	0	$0.2 \times 3 + 0.8 \times 100 = 80.6$
0	1	$0.01 \times 0 + 0.99 \times 80 = 79.2$
1	0	$0.2 \times 35 + 0.8 \times 95 = 83$
1	1	$0.01 \times 30 + 0.99 \times 75 = 74.55$

Representation 2: Single-stage decision network

Definition

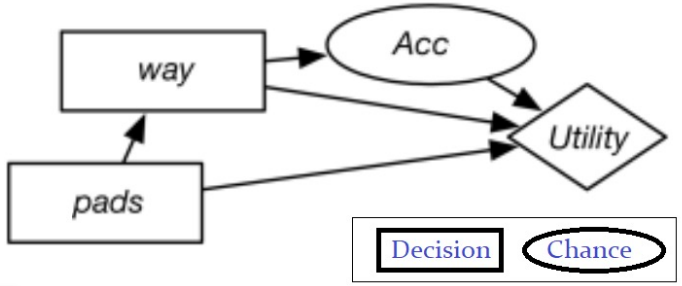
A single-stage decision network (SSDN) is an extension of a Bayesian network with three kinds of nodes:

- $D_1, \ldots, D_k$: the decision nodes, representing the decision variables. If there are multiple decision nodes, then they are arranged in a total ordering, such that the parents of a decision node D_i are $D_1, \ldots, D_{i-1}$
- $C_1, \ldots, C_l$: the chance nodes, representing random variables. Each chance node has a conditional probability given its parents.
- A single utility node, representing the utility.

Note: The chance nodes take the same role as nodes in a Bayesian network with local Markov property

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

Example. [delivery robot]





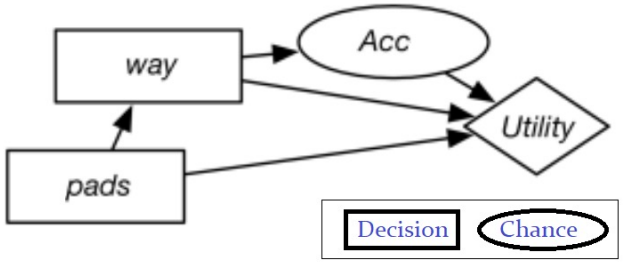
Definition

- A policy for a single-stage decision network is an assignment of a value to each decision variable. Each policy has an expected utility.
- The optimal policy is a policy whose expected utility is maximal.
- The value of a decision network is the expected utility of an optimal policy of the network.

The SSDN-based one-off decision problem asks for the optimal policy and the value of a given SSDN

We can extend the CPT operations to the set of CPT and the utility function table:

Example. [delivery robot]





Way	Acc	P(Acc | Way)
0	0	0.8
0	1	0.2
1	0	0.99
1	1	0.01

×

Pad	Way	Acc	u(Pad, Way, Acc)
0	0	0	100
0	0	1	3
0	1	0	80
0	1	1	0
1	0	0	95
1	0	1	35
1	1	0	75
1	1	1	30

=

Pad	Way	Acc	f(Pad, Way, Acc)
0	0	0	0.8×100
0	0	1	0.2×3
0	1	0	0.99×80
0	1	1	0.01×0
1	0	0	0.8×95
1	0	1	0.2×35
1	1	0	0.99×75
1	1	1	0.01×30

Acc-Sum:

Pad	Way	E(u | Pad, Way)
0	0	0.2×3+0.8×100=80.6
0	1	0.01×0+0.99×80=79.2
1	0	0.2×35+0.8×95=83
1	1	0.01×30+0.99×75=74.55

Note. For convenience, we refer to the utility function table as "CPT" as well.

We can extend the variable elimination algorithm for Bayesian networks to SSDN.

VariableEliminationSSDN($SSDN$)

INPUT: *SSDN*

OUTPUT: An optimal policy and the value of *SSDN*

Prune all nodes that are not ancestors of the utility node
for every chance node C **do**
Multiply all CPTs that contains C
Perform summation over C to eliminate C
end for
Let v be the maximum value in the last CPT.
Let d be the assignments that gives the maximum value
return d, v

Note.

- No chance node can be a parent of a decision node
- Decisions are made without making any observation

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

Sequential Decision Problems

For sequential decision problem, we use two types of representations:

- Finite horizon: Decision network
- Indefinite horizon or infinite horizon: Markov decision process.

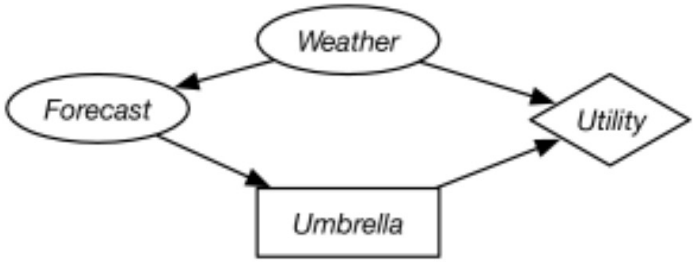
Definition

A decision network (DN) is a probabilistic graphical model that extends SSDN to allow both chance nodes and decision nodes to be parents of decision nodes.

- The graph is acyclic.
- $D_1, \dots, D_k$: the decision nodes
- $C_1, \dots, C_l$: the chance nodes
- If an edge connects a node X to D_i, then there is an edge from X to all nodes D_j where $j > i$. (No forgetting property)
- A CPT $P(C_i|\text{parents}(C_i))$ is assigned to every chance node C_i
- A utility function $u(\text{parents}(u))$ is given

Example. [weather] Should the agent take an umbrella?

- Chance node: Weather: {norain, rain}, Forecast: {sunny, cloudy, rainy}
- Decision node: Umbrella:{0,1}
- The problem has the following decision network



Weather	Umbrella	u(Weather, Umbrella)
norain	0	20
norain	1	100
rain	0	70
rain	1	0

Weather	P(Weather)
norain	0.7
rain	0.3

Weather	Forecast	P(Forecast|Weather)
norain	sunny	0.7
norain	cloudy	0.2
norain	rainy	0.1
rain	sunny	0.15
rain	cloudy	0.25
rain	rainy	0.6

There are three types of edges:

- Into a decision node: available information (e.g., observation) when the decision is made.
- Into a chance node: probabilistic dependence
- Into the utility node: dependence of the utility function.

Definition

- A decision function for a decision node D_i is a function

$$d_i: \prod_{D_j \in \text{parents}(D_i)} \text{dom}(D_j) \rightarrow \text{dom}(D_i)$$

  which maps all possible values of $\text{parents}(D_i)$ to $\text{dom}(D_i)$
- A policy π consists of a decision function for every decision node $D_1, \ldots, D_k$
- A state ω satisfies policy π if every decision node has the same value as specified by π, given the values of its parents in ω.
- The expected utility of policy π is

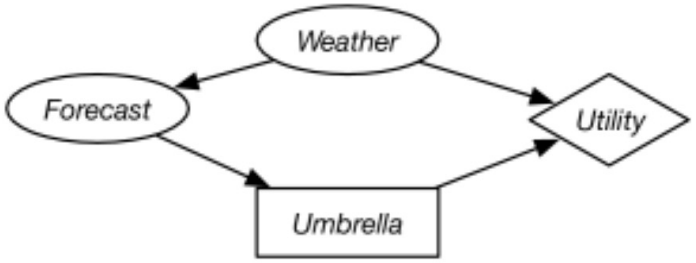
$$E(u|\pi) = \sum_{\omega \text{ satisfies } \pi} u(\omega) \times P(\omega)$$

The DN-based sequential decision problem asks for the optimal policy (which maximises the expected utility) and the value of a given decision network.

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

Example. [weather] A policy is a function π: {sunny, cloudy, rainy} → {0,1}

- Always bring the umbrella $\pi(\text{sunny}) = \pi(\text{cloudy}) = \pi(\text{rainy}) = 1$
- Bring the umbrella only if the forecast is rainy $\pi(\text{rainy}) = 1, \pi(\text{sunny}) = \pi(\text{cloudy}) = 0$



Weather	Umbrella	u(Weather, Umbrella)
norain	0	100
norain	1	20
rain	0	0
rain	1	70

Suppose π_1: $\pi_1(\text{cloudy}) = 1, \pi_1(\text{sunny}) = \pi_1(\text{rainy}) = 0$

Weather	Forecast	Umbrella	$u(s)\times P(s)$
norain	sunny	0	P(norain)P(sunny | norain)u(norain, 0) = 0.7×0.7×100
norain	cloudy	1	P(norain)P(cloudy | norain)u(norain, 1) = 0.7×0.2×20
norain	rainy	0	P(norain)P(rainy | norain)u(norain, 0) = 0.7×0.1×100
rain	sunny	0	P(rain)P(sunny | rain)u(rain, 0) = 0.3×0.15×0
rain	cloudy	1	P(rain)P(cloudy | rain)u(rain, 1) = 0.3×0.25×70
rain	rainy	0	P(rain)P(rainy | rain)u(rain, 0) = 0.3×0.6×0

Weather	P(Weather)
norain	0.7
rain	0.3

Weather	Forecast	P(Forecast|Weather)
norain	sunny	0.7
norain	cloudy	0.2
norain	rainy	0.1
rain	sunny	0.15
rain	cloudy	0.25
rain	rainy	0.6

Expected utility:

$$E(u|\pi_1) = 49 + 2.8 + 7 + 0 + 5.25 + 0 = 64.05$$

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

We can extend the variable elimination algorithm for SSDN to decision networks.

Idea: Repeatedly eliminate decisions

- Start from the last decision node, find an optimal decision for this node
- Reduce this decision node

VariableEliminationDN(DN)

INPUT: DN
OUTPUT: An optimal policy and the value of DN

Create a set DF of decision functions, initially $\emptyset$
Remove all variables that are not ancestors of the utility node
while there are decision nodes remaining **do**
    Eliminate each random variable that is not a parent of a decision node.
    Let D be the last decision node, multiply all tables that contains D
    Add $\text{argmax}_D F_u$ to DF
end while
return DF and the product of the remaining tables

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

Example. [weather]

- Eliminate weather

Weather	P(Weather)
norain	0.7
rain	0.3

×

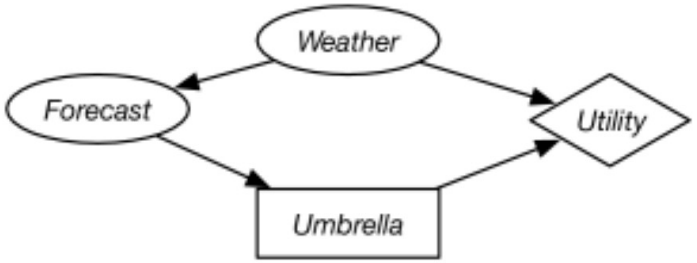
Weather	Forecast	P(Forecast|Weather)
norain	sunny	0.7
norain	cloudy	0.2
norain	rainy	0.1
rain	sunny	0.15
rain	cloudy	0.25
rain	rainy	0.6

×

Weather	Umbrella	u(Weather, Umbrella)
norain	0	100
norain	1	20
rain	0	0
rain	1	70

⇒

Umbrella	Forecast	F_u
0	sunny	49
1	sunny	12.95
0	cloudy	14
1	cloudy	8.05
0	rainy	7
1	rainy	14



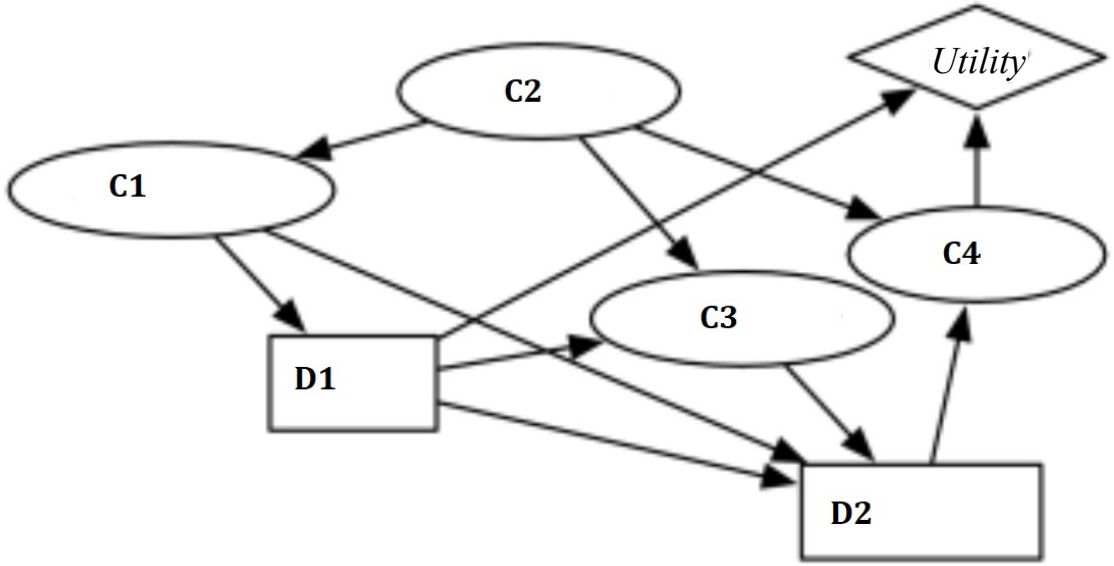
- Eliminate umbrella

Forecast	F_u
sunny	49
cloudy	14
rainy	14

Optimal policy: $\pi^*(\text{sunny}) = \pi^*(\text{cloudy}) = 0, \pi^*(\text{rainy}) = 1$

Expected utility: $E(u|\pi^*) = 49 + 14 + 14 = 77$

Example. A DN with two decision nodes:



- Order of elimination:
  - First iteration: C_2, C_4, D_2
  - Second iteration C_3, D_1

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

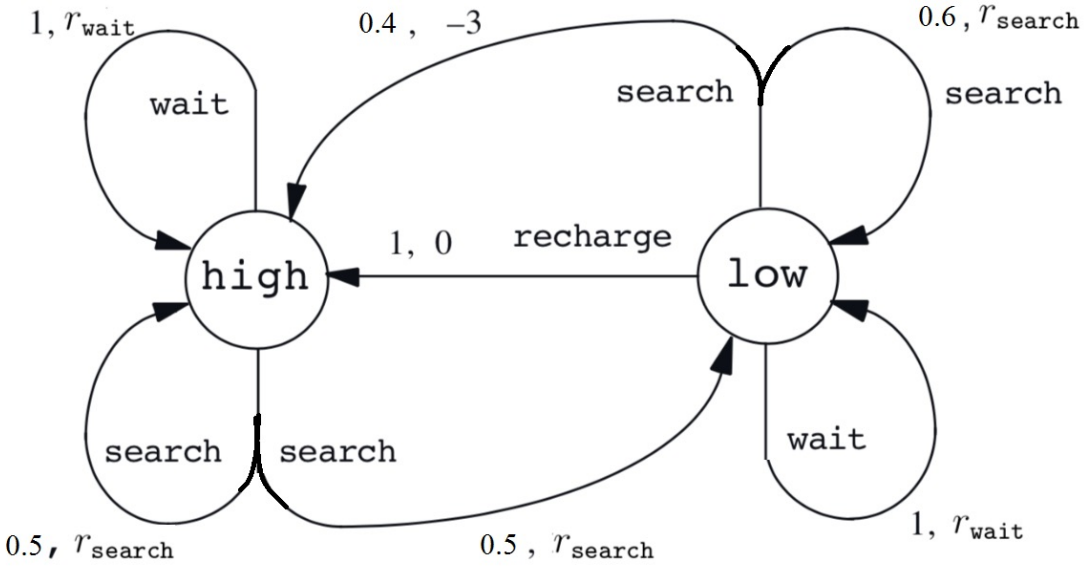
Markov Decision Process

Markov Decision Process

Example. A robot that searches for items

- The robot is battery-powered and can be in two states: high, low.
- With certain probability, the robot will power-off and need costly manual restart.
- At every time step, the agent makes a decision between waits, searches, and recharge.





Definition

A Markov decision process (MDP) is a 5-tuple $M = (S, A, P, R, \gamma)$ where

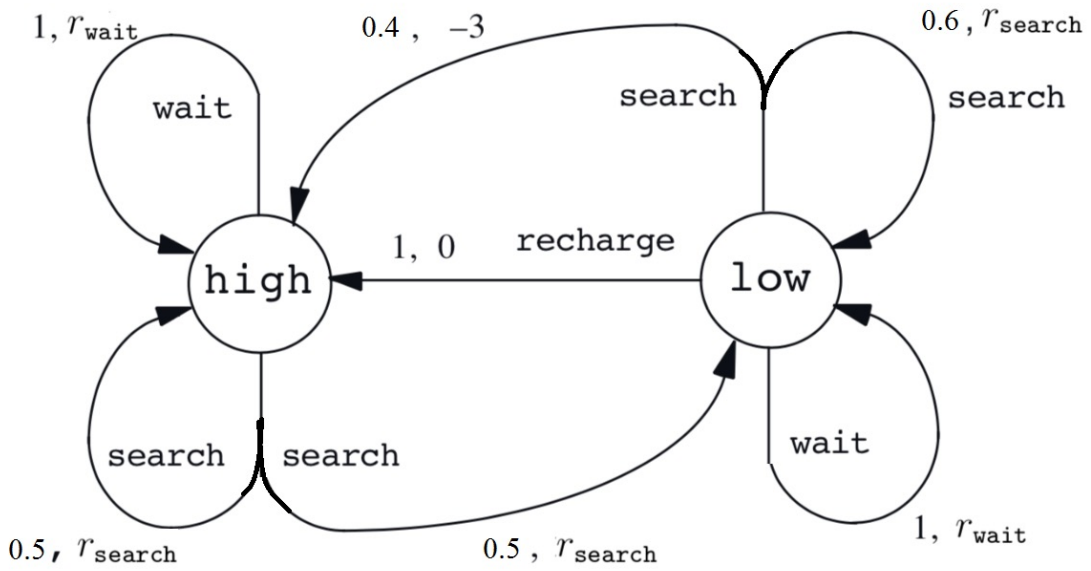
- S: the state space
- A: the action space
- $P: S \times A \times S$ is a transition probability function, with $P(s, a, s')$ be the probability of a transition from state s to state s' by performing action a. We denote this as $P^a_{s \to s'}$ for short.
- $R: S \times A \times S \to \mathbb{R}$ is a reward function, with $R(s, a, s')$ be the immediate reward the agent receives from a transition from s to s' by performing action a. We denote this as $R^a_{s \to s'}$ for short
- $\gamma \in (0,1)$ is a discount factor.

A run of an MDP is a (finite or infinite) sequence $s_0, a_1, r_1, s_1, a_2, r_2, s_2, a_3, r_3, s_3, \ldots$ such that every $s_i \in S$, every $a_i \in A$ and $r_{i+1} = R^{a_{i+1}}_{s_i \to s_{i+1}}$

Example.

- States: $S = \{H, L\}$
- Actions: $A = \{\text{search}, \text{wait}, \text{recharge}\}$
- Transition probability and reward are given by the table below:

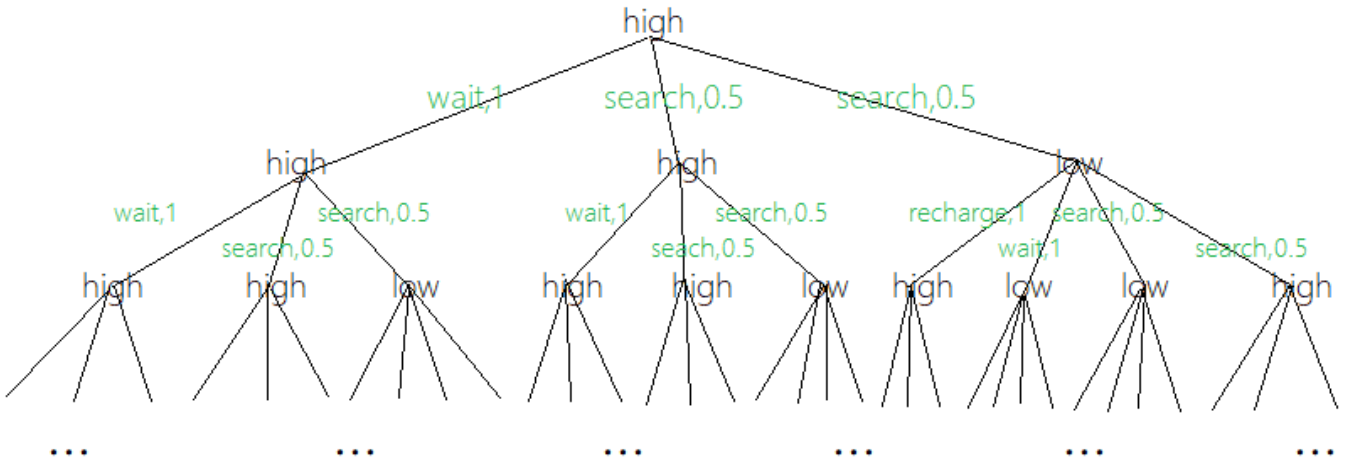
s	a	s'	$p(s,a,s')$	$r(s,a,s')$
high	search	high	0.5	r_{search}
high	search	low	0.5	r_{search}
low	search	high	0.4	-3
low	search	low	0.6	r_{search}
high	wait	high	1	r_{wait}
high	wait	low	0	r_{wait}
low	wait	high	0	r_{wait}
low	wait	low	1	r_{wait}
low	recharge	high	1	0
low	recharge	low	0	0.



An example run: high, wait, r_{wait}, high, search, r_{search}, low, recharge, 0, high

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

The decision tree that represents runs of an MDP:



- The transition probabilities represent the belief of the agent about the environment: Given the agent is in a state s, assuming an action a, the chance of moving into a specific state in the next stage.
- The sequence of decisions to be made is not fixed, depending on the dynamics of environment, i.e., the transition probabilities $P^{a}_{s \to s'}$ and the rewards $R^{a}_{s \to s'}$
- In principle, the agent's behaviour depends on the entire history of run.
- However, we typically assume that a MDP also generates a stochastic process that satisfies the Markov property, i.e., the future actions depend only upon the present state, not on the history that preceded it.

Policy and Utility

Definition

- A policy of an MDP is a function $\pi: S \to A$ such that $\pi(s)$ indicates the action to be taken at state s.
- Given a run $s_0, a_1, r_1, s_1, \ldots, a_T, r_T, s_T$, the γ-discounted utility is $\sum_{t=0}^{T} \gamma^t \cdot r_{t+1}$. When a run is infinite, i.e., $T = \infty$, $\sum_{t=0}^{\infty} \gamma^t \cdot r_{t+1}$.
- For a policy π and a starting state $s \in S$, the value $V^{\pi}(s)$ is defined as the expected utility:

$$V^{\pi}(s) = E_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t \cdot r_{t+1} \,|s_0 = s\right]$$

Policy Evaluation

Policy evaluation takes an MDP and a policy π as input, and ask for a value function that expresses the cumulative rewards of runs determined by π

$$V^{\pi}(s) = E_{\pi}\left[\sum_{t=0}^{\infty} \gamma^{t} \cdot r_{t+1} \,|s_0 = s\right]$$

Note that we have:

$$V^{\pi}(s) = E_{\pi}\left[\sum_{t=0}^{\infty} \gamma^{t} \cdot r_{t+1} \,|s_0 = s\right] = \sum_{s' \in S} P_{s \to s'}^{\pi(s)} \left(R_{s \to s'}^{\pi(s)} + \gamma V^{\pi}(s')\right)$$

Definition

Bellman expectation equation

Let $R_s^a = \sum_{s' \in S} P_{s \to s'}^{a} R_{s \to s'}^{a}$ for all $a \in A$

$$V^{\pi}(s) = R_s^{\pi(s)} + \gamma \sum_{s' \in S} P_{s \to s'}^{\pi(s)} V^{\pi}(s')$$

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

Algorithm for policy evaluation: Iteratively update $V^{\pi}(s)$ with dynamic programming

- Let $V_k^{\pi}(s)$ denote the expected discounted accumulative reward for any run of length k, staring from s.
- Iteration: Apply Bellman expectation equation:

$$V_{k+1}^{\pi}(s) = R_s^{\pi(s)} + \gamma \sum_{s' \in S} P_{s \to s'}^{\pi(s)} V_k^{\pi}(s')$$

- Convergence: $V^{\pi} := \lim_{k \to \infty} V_k^{\pi}$

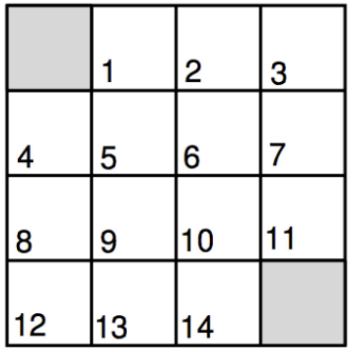
EvaluatePolicy$((S, A, P, R, \gamma), \pi)$

INPUT: An MDP (S,A,P,R,γ), a policy π
OUTPUT: The expected reward function V^{π}

Set $V(s) \leftarrow 0$ for all $s \in S$
repeat
  for $s \in S$ **then**
    $V'^{\pi}(s) \leftarrow R_s^{\pi(s)} + \gamma \sum_{s' \in S} P_{s \to s'}^{\pi(s)} V^{\pi}(s')$
  end for
  Set $V^{\pi} \leftarrow V'^{\pi}$
Until V^{π} converges
Return V^{π}

Example [Gridworld]. An agent traversing a grid in order to get to two target cells (shaded)

- The agent sits in one cell at a time, i.e., states $S = \{0,1,\dots,14,15\}$
- $A = \{\uparrow, \downarrow, \leftarrow, \rightarrow\}$. Actions leading out of the grid will not change the position
- State transitions are all deterministic
- Reward is -1 for any action until a target is reached
- Suppose π is the random policy $\pi(s,\uparrow) = \pi(s,\downarrow) = \pi(s,\leftarrow) = \pi(s,\rightarrow) = 1/4$.



Apply EvaluatePolicy with $\gamma = 0.4$

$k = 0$

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

$k = 1$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

$k = 2$

0.0	-1.3	-1.4	-1.4
-1.3	-1.4	-1.4	-1.4
-1.4	-1.4	-1.4	-1.3
-1.4	-1.4	-1.3	0.0

$k = 3$

0.0	-1.41	-1.55	-1.56
-1.41	-1.54	-1.56	-1.55
-1.55	-1.56	-1.54	-1.41
-1.56	-1.55	-1.41	0.0

$$V^{\pi}(s) = R_s^{\pi(s)} + \gamma \sum_{s' \in S} P_{s \to s'}^{\pi(s)} V^{\pi}(s')$$

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

Decision Problem

The MDP-based decision problem takes an MDP as input and asks for a policy that maximises the cumulative rewards of samples.

MDP-based decision problem

INPUT: MDP (S,A,P,R,γ), initial state s

OUTPUT: Optimal policy $\pi^*: S \to A$ where

$$\pi^* = \text{argmax}_\pi V^\pi(s)$$

Algorithms for this problem:

- Policy iteration
- Value iteration

Definition

Define the action value function $Q: S \times A \to \mathbb{R}$ to evaluate the expected reward for performing an action a at state s:

$$Q(s,a) = E\left[\sum_{t=0}^{\infty} \gamma^t \cdot r_{t+1} \,|s_0 = s, a_0 = a\right],$$

where r_t is the immediate reward received at stage t.

- By definition of V^π, we have

$$Q(s,a) = R_s^a + \gamma \sum_{s' \in S} P_{s \to s'}^a \cdot V^\pi(s')$$

- This means that

$$V^\pi(s) = Q(s, \pi(s))$$

- Let π^* be the optimal policy, we have

$$V^*(s) = V^{\pi^*}(s) = \max_{a \in A} Q(s,a)$$

Bellman optimal action equation

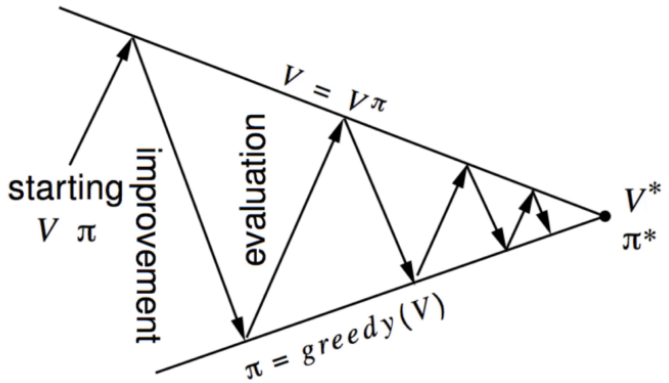
$$Q(s,a) = R_s^a + \gamma \sum_{s' \in S} P_{s \to s'}^a \cdot \max_{a' \in A} Q(s', a')$$

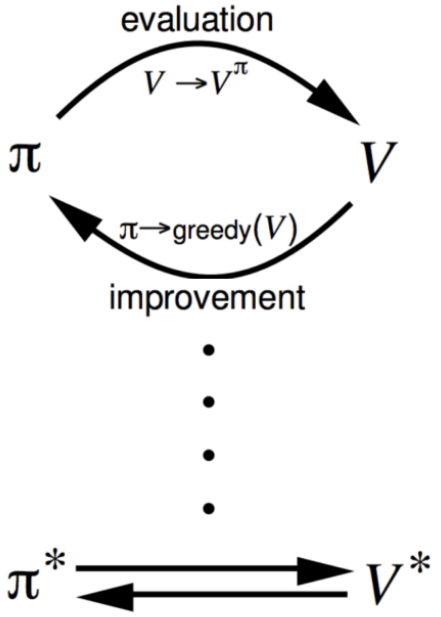
Algorithm 1: Policy Iteration

We aim to improve a given policy iteratively:

- Evaluate the policy π to produce the valuation function V^{π}
- Improve the policy by acting greedily according to Q.

Repeat the process until π converges.





哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

ImprovePolicy$((S, A, P, R, \gamma), \pi, V^{\pi})$

INPUT: An MDP (S,A,P,R,γ), a policy π, a value function V^{π}

OUTPUT: A policy π'

for $s \in S$ **do**

  for $a \in A$ **then**

    $Q(s,a) \leftarrow R_s^a + \gamma \sum_{s' \in S} P_{s \to s'}^a V^{\pi}(s')$

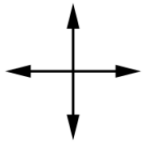
  end for

  $\pi'(s) \leftarrow \text{argmax}_{a \in A} Q(s,a)$

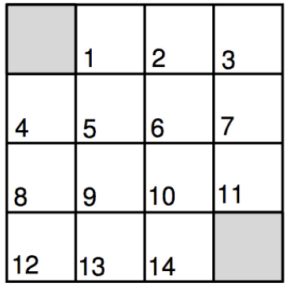
end for

return π'

Example. [Gridworld]



actions



$r = -1$
on all transitions

v_k for the Random Policy

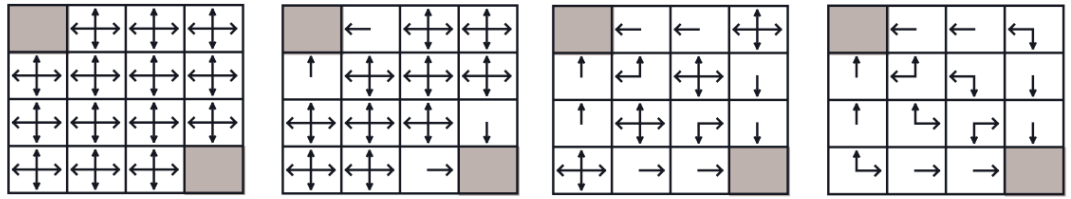
$k=0$			
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

$k=1$			
0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

$k=2$			
0.0	-1.3	-1.4	-1.4
-1.3	-1.4	-1.4	-1.4
-1.4	-1.4	-1.4	-1.3
-1.4	-1.4	-1.3	0.0

$k=3$			
0.0	-1.41	-1.55	-1.56
-1.41	-1.54	-1.56	-1.55
-1.55	-1.56	-1.54	-1.41
-1.56	-1.55	-1.41	0.0

Greedy Policy



哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

The policy iteration algorithm computes the optimal policy for a given MDP.

PolicyIteration$((S, A, P, R, \gamma))$

INPUT: An MDP (S,A,P,R,γ)
OUTPUT: Optimal policy π^*

Set $V(s) \leftarrow 0$ for all $s \in S$
Set initial $\pi(s)$ for all $s \in S$
repeat
$V \leftarrow$EvaluatePolicy$((S, A, P, R, \gamma), \pi)$
$\pi \leftarrow$ImprovePolicy$((S, A, P, R, \gamma), \pi, V)$
until π converges
return π

Note. The output of the policy iteration algorithm satisfies Bellman optimal action equation:

$$Q(s,a) = R_s^a + \gamma \sum_{s' \in S} P_{s \to s'}^a \cdot \max_{a' \in A} Q(s', a')$$

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

Algorithm 2: Value Iteration

Problem of policy iteration: In policy evaluation, convergence to V^{π} only happens in the limit.

Is it possible to obtain the optimal policy without evaluating every intermediate policy?

Optimality

A policy π achieves the optimal value from state s if and only if,

$$\pi(s) = \text{argmax}_{a \in A} \sum_{s' \in S} P^{a}_{s \to s'} (R^{a}_{s \to s'} + \gamma V^{*}(s')):$$

- If we can compute V^{*}, the optimal policy can be identified.
- Value iteration aims to evaluate V^{*} without evaluating the policy

We now derive V^*:

$$
\begin{aligned}
V^*(s) &= \max_{a \in A} Q(s,a) \\
&= \max_{a \in A} \left(R_s^a + \gamma \sum_{s' \in S} P_{s \to s'}^a \cdot \max_{a' \in A} Q(s', a') \right) \\
&= \max_{a \in A} \left(\sum_{s' \in S} P_{s \to s'}^a R_{s \to s'}^a + \gamma \sum_{s' \in S} P_{s \to s'}^a \cdot \max_{a' \in A} Q(s', a') \right) \\
&= \max_{a \in A} \left(\sum_{s' \in S} P_{s \to s'}^a \cdot \left(R_{s \to s'}^a + \gamma \max_{a' \in A} Q(s', a') \right) \right) \\
&= \max_{a \in A} \left(\sum_{s' \in S} P_{s \to s'}^a \cdot \left(R_{s \to s'}^a + \gamma V^*(s') \right) \right)
\end{aligned}
$$

Bellman optimal value equation

$$
V^*(s) = \max_{a \in A} \sum_{s' \in S} P_{s \to s'}^a \cdot \left(R_{s \to s'}^a + \gamma V^*(s') \right)
$$

The value iteration algorithm iteratively computes the optimal value for a state, then find the action that maximises the expected value

ValueIteration$((S, A, P, R, \gamma))$

INPUT: An MDP (S,A,P,R,γ)

OUTPUT: Optimal policy π^*

Set $V(s) \leftarrow 0$ for all $s \in S$

repeat

for $s \in S$ **then**

$V'(s) \leftarrow \max_{a \in A} \sum_{s' \in S} P^a_{s \to s'} \cdot \left(R^a_{s \to s'} + \gamma V(s')\right)$

end for

Set $V \leftarrow V'$

Until V converges

$\pi(s) \leftarrow \operatorname{argmax}_{a \in A} \sum_{s' \in S} P^a_{s \to s'} \cdot \left(R^a_{s \to s'} + \gamma V(s')\right), \forall s \in S$

return π

Summary

The following are the main knowledge points covered

- The utility theory?
- One-off decision problem
  - Decision Tree and Single-Stage Decision Network
- Sequential decision problem
  - Markov Decision Process (MDP)
  - Policy iteration and value iteration algorithm for solving MDP